SYSTEM DESIGN — SESSION 1

HLD + LLD Foundations

Kashyap Pandey | Apr 4, 2026 | Principal Engineer Prep

CAP Theorem • 8 Building Blocks • HLD Template • LLD Template

1. Distributed Systems + CAP Theorem

KEY: *You can only guarantee 2 of 3: Consistency, Availability, Partition Tolerance. Partition WILL happen → real choice is always C vs A.*

What Is A Distributed System?

Multiple machines working together to appear as ONE system to the end user. Needed when one machine cannot handle the traffic, storage, or availability requirements.

- Vertical scaling (bigger machine) has physical limits
- Horizontal scaling (more machines) = distributed system
- Challenge: consistency, availability, partition tolerance cannot all be guaranteed

CAP Theorem

Property	Meaning	Example
Consistency	Every read gets the most recent write	Bank balance — always accurate
Availability	Every request gets a response (no timeout)	System never goes down
Partition Tolerance	System works even when network fails between nodes	Network split → still operates

CP vs AP — When To Choose

Choose CP — Consistency Priority	Choose AP — Availability Priority
• Banking, payments, financial data • Inventory (last item in stock) • HBase, MongoDB, Zookeeper • *"Wrong data = real money lost"*	• Social media feeds, likes, views • Shopping cart, DNS, messaging • Cassandra, DynamoDB, CouchDB • *"Stale data for 100ms is acceptable"*

Consistency Models — The Spectrum

- Strong Consistency: read always returns latest write. Expensive. Use: bank balance.
- Eventual Consistency: reads may be stale but converge. Cheap. Use: Facebook likes.
- Read-Your-Writes: YOU always see your own write. Others may not. Use: Twitter own tweets.
- Monotonic Reads: once you see a value, never see older. Use: WhatsApp message history.

---

2. The 8 Building Blocks

> **KEY:** *Mnemonic: 'Load Databases Carefully. Make Cool APIs. Often Search Everything.' — Every system design uses combinations of these 8 blocks.*

#	Block	One-Line Role	When To Use
1	**Load Balancer**	Distribute traffic. No server overwhelmed.	RPS > 10K, need redundancy, horizontal scale
2	**Database**	Persist structured data. SQL=relations. NoSQL=scale.	Always needed. SQL for ACID. NoSQL for scale/flex.
3	**Cache**	In-memory fast reads. 100x faster than DB.	Read:Write > 3:1. Hot data. Session storage.
4	**Message Queue**	Async decoupling. Producer won't wait for consumer.	Long tasks, retry needed, service decoupling.
5	**CDN**	Static files near users globally.	Images, videos, CSS, JS. Global users.
6	**API Gateway**	Single entry. Auth + rate limit + routing.	Multiple services, mobile clients, public APIs.
7	**Object Storage**	Binary files. Images, videos, backups.	Files > 1MB. S3-style. Not DB.
8	**Search Index**	Full text search. Not SQL LIKE.	User searches text. Autocomplete. Fuzzy match.

Cache Strategies — Quick Reference

Strategy	Flow	Use When
Cache Aside	Check cache → miss → DB → write to cache	Most common. Read-heavy. App controls cache.
Write Through	Write to cache AND DB simultaneously	Must be consistent. Slower writes.
Write Back	Write to cache only → async write to DB	Highest write speed. Risk of data loss.

3. HLD 7-Step Template

> 💡 **KEY:** *Mnemonic: 'Really Serious APIs Don't Handle Bad Traffic' — Use same order every interview. Never draw boxes before clarifying requirements.*

#	Step	What To Do	What To Say
R	**Requirements**	Functional (what it does) + Non-functional (how it behaves)	*"Let me clarify requirements first. Core features are... Non-functional: latency, scale, availability..."*
S	**Scale Estimation**	DAU × requests = RPS. Records × size = storage. Read:Write ratio → architecture	*"Let me estimate scale. Given X users × Y requests = Z RPS. Storage = ..."*
A	**API Design**	Method + endpoint + request body + response + auth	*"Key APIs are: POST /shorten body={longUrl} → returns shortUrl..."*
D	**Data Model**	Entities + attributes. SQL vs NoSQL + reason. Primary + secondary stores	*"Data model: one table with shortCode PK, longUrl, userId. Using DynamoDB because..."*
H	**High Level Design**	Draw boxes: LB, API GW, services, DB, cache. Show read + write paths	*"High level: Client → LB → API Gateway → App Server → Redis/DB..."*
B	**Bottlenecks**	DB overload, SPOF, hot spots, cascading failures — and fixes	*"Main bottleneck: 100:1 ratio → cache 80% of reads. Hot key: replicate in Redis..."*
T	**Trade-offs**	Chose X because Y. Gave up Z. Acceptable because W.	*"I chose 302 over 301 because analytics required. Trade-off: more server hits. Acceptable because..."*

Scale Estimation — 3 Numbers Always Calculate

```
1. RPS = DAU × requests_per_day ÷ 86,400
   Read:Write ratio → drives caching/replication decision

2. Storage = records_per_day × bytes_per_record × retention_days
   > 1TB per node → need sharding

3. Bandwidth = RPS × response_size_bytes
   > 1Gbps → need CDN or edge caching

Key thresholds:
  RPS > 10K       → multiple servers + LB
  Read:Write > 5:1 → read replicas + cache
  Storage > 1TB   → sharding needed
  Users global    → CDN + multi-region
```

5 Common Bottlenecks + Fixes

Bottleneck	Symptom	Fix
Database overload	DB CPU 100%, slow queries	Read replicas, caching, sharding

Single point of failure	One crash = system down	Redundancy, multiple instances, LB
Hot spot / hot key	One shard gets all traffic	Local cache, replicate hot data
Network latency	Cross-region calls slow	CDN, edge caching, data locality
Cascading failures	One service down → chain reaction	Circuit breaker, timeouts, bulkhead

---

4. LLD 6-Step Template

> **KEY:** *Mnemonic: 'Every Relationship Becomes A Pattern In Java' — Always define interfaces before implementations. Ask 'does this have one job?' for every class.*

#	Step	What To Do	Example
E	**Entities**	Identify real-world nouns that have data or behaviour	ParkingLot, Floor, Spot, Vehicle, Ticket, Payment
R	**Relationships**	IS-A (inheritance), HAS-A (composition), USES-A (dependency)	ParkingLot HAS-A Floor[]. Car IS-A Vehicle.
B	**Behaviours**	Identify verbs → methods for each entity	Spot: assign(), free(), isAvailable(). Lot: park(), remove()
A	**Apply Pattern**	Which design pattern fits? (see patterns below)	Strategy for payment. Singleton for ParkingLot. Observer for waitlist.
I	**Interfaces**	Define contracts before implementations	interface PaymentProcessor { Receipt process(Ticket t); }
J	**Java Classes**	Write actual code starting with simplest version	class ParkingSpot { int id; boolean occupied; void assign(Vehicle v){...} }

5 Design Patterns — Must Know

Pattern	What It Does	Use When	Example
Singleton	Only ONE instance ever	Config, Logger, ParkingLot	*ParkingLot.getInstance()*
Factory	Create objects without specifying exact class	Different vehicle types, payment types	*VehicleFactory.create("CAR")*
Strategy	Swap algorithms at runtime	Payment methods, code generation, sorting	*PaymentStrategy: CreditCard/UPI/Cash*
Observer	One change → notify all dependents	Seat booked → notify waitlist, event systems	*Show.notifyAll(seatBookedEvent)*
Decorator	Add behaviour without changing class	Logging, caching, auth wrapping	*CachedURLService wraps URLService*

SOLID Principles — One Line Each

	Principle	Rule	Violation Example
S	**Single Responsibility**	One class = one job	ParkingLot calculating payment fees
O	**Open/Closed**	Open for extension, closed for modification	Editing existing class to add new vehicle type

L	**Liskov Substitution**	Subclass must work where parent is expected	ElectricCar breaks Car's contract
I	**Interface Segregation**	Don't force unused method implementations	Parking + Tax in one interface
D	**Dependency Inversion**	Depend on interfaces not implementations	ParkingLot depending on CreditCardProcessor directly

5. Applied Example — URL Shortener

> 💡 **KEY:** *Two flows: POST /shorten → generate code → store in DB. GET /{code} → Redis check → DB fallback → 302 redirect. Analytics async via MQ.*

HLD — Architecture Diagram

```
WRITE FLOW (shorten a URL):
  Client → Load Balancer → API Gateway → App Server
    → codeGenerator.generate(longUrl)
    → urlRepository.save(shortCode, longUrl, userId)
    → return 'https://bit.ly/' + shortCode

READ FLOW (redirect):
  Client → Load Balancer → API Gateway → App Server
    → cacheService.get(shortCode)
      HIT  → return longUrl → HTTP 302 redirect
      MISS → urlRepository.findByShortCode(shortCode)
           → cacheService.put(shortCode, longUrl)
           → return longUrl → HTTP 302 redirect
    → async: push clickEvent → Message Queue
                             → Analytics Service
                             → Analytics DB
```

Scale Estimation

```
Given: 100M URLs/day (writes), 10B redirects/day (reads)
       500 bytes per URL, keep 5 years

Write RPS = 100M ÷ 86,400 = ~1,157 RPS
Read  RPS = 10B  ÷ 86,400 = ~115,740 RPS
Read:Write = 100:1 → HEAVILY READ → Redis cache essential

Storage/day = 100M × 500 bytes = 50 GB/day
5 years     = 50 GB × 365 × 5 = ~91 TB total
91 TB → cannot fit one machine → SHARDING needed
```

Key Design Decisions

Decision	Chose	Gave Up	Why Acceptable
301 vs 302	302 (temporary)	Slightly more server hits	Analytics tracked on every click. 301 bypasses server.
SQL vs NoSQL	DynamoDB (NoSQL)	Complex queries, ACID	Pure key lookup. 91TB needs horizontal sharding.

Sync vs Async analytics	Async via MQ	Real-time accuracy	Redirect must be <50ms. 1-2s analytics delay acceptable.
Short code generation	Base62 of auto-ID	Predictable sequence	Guaranteed unique. Fast. Add random salt if needed.

LLD — URLShortenerService

```
public class URLShortenerService {
    private final URLRepository urlRepository;
    private final CacheService  cacheService;
    private final CodeGenerator codeGenerator;  // Strategy pattern

    // Dependency Injection — depends on interfaces not implementations (D in SOLID)
    public URLShortenerService(URLRepository repo, CacheService cache, CodeGenerator
gen) {
        this.urlRepository = repo;
        this.cacheService   = cache;
        this.codeGenerator  = gen;
    }

    public String shorten(String longUrl, String userId) {
        String shortCode = codeGenerator.generate(longUrl);       // generate
        urlRepository.save(shortCode, longUrl, userId);           // persist
        return "https://bit.ly/" + shortCode;                   // return full URL
    }

    public String redirect(String shortCode) {
        String longUrl = cacheService.get(shortCode);             // check cache
        if (longUrl != null) return longUrl;                      // cache HIT
        longUrl = urlRepository.findByShortCode(shortCode);       // cache MISS → DB
        if (longUrl == null) throw new NotFoundException();       // 404
        cacheService.put(shortCode, longUrl);                     // write back
        return longUrl;                                           // caller does 302
    }
}

// Interfaces (I in SOLID — depend on contracts not implementations):
interface CodeGenerator  { String generate(String longUrl); }
interface URLRepository  { void save(String code, String url, String userId);
                           String findByShortCode(String code); }
interface CacheService   { String get(String key); void put(String key, String val); }
```